7 High-Dimensional Application: ANN Training

7.1 Motivation

The benchmarks in Sections 3 to 5 have at most 100 design variables and a known global minimum. They are well suited for following the path of the optimizer, but practical problems in engineering and machine learning usually have thousands to millions of unknowns. In this section, we will explore the training of an ANN (Artificial Neural Network) with respect to different ADAM variants.

Even the moderate network used for the 100-dimensional problem below, with layer widths $[100, 128, 64, 1]$, has

$$P = \sum_{l=1}^{L} n_l \, (n_{l-1} + 1) = 12\,928 + 8\,256 + 65 = 21\,249 \tag{29}$$

trainable parameters. The gradient with respect to all of them is obtained with the adjoint method (backpropagation) at a cost comparable to one forward pass. A central finite-difference approximation would need $2P = 42\,498$ forward passes per gradient. In addition, the loss of a neural network has no single known minimizer. Performance is measured by how well the trained model predicts data it has not seen, which is a different criterion from the distance to w^* used in Section 5.

7.2 Network Model

The network maps an input $\boldsymbol{x} \in \mathbb{R}^{n_0}$ to an output $\boldsymbol{y} \in \mathbb{R}^{n_L}$ through L fully connected layers with widths $[n_0, n_1, \ldots, n_L]$. Each layer has a weight matrix $\boldsymbol{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ and a bias vector $\boldsymbol{b}^{(l)} \in \mathbb{R}^{n_l}$. The samples are stored column-wise, $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, so that one forward pass evaluates a whole batch:

$$\boldsymbol{A}^{(0)} = \boldsymbol{X}, \tag{30}$$

$$\boldsymbol{Z}^{(l)} = \boldsymbol{W}^{(l)} \boldsymbol{A}^{(l-1)} + \boldsymbol{b}^{(l)} \mathbf{1}^{\mathsf{T}}, \qquad l = 1, \ldots, L, \tag{31}$$

$$\boldsymbol{A}^{(l)} = \sigma\big(\boldsymbol{Z}^{(l)}\big) \quad (l < L), \qquad \boldsymbol{A}^{(L)} = \boldsymbol{Z}^{(L)}. \tag{32}$$

The hidden layers use the activation function σ element-wise, and the output layer is linear. The training objective is the mean squared error over all outputs and all N_b samples of the current batch,

$$J(\boldsymbol{\theta}) = \frac{1}{n_L N_b} \left\| \boldsymbol{A}^{(L)} - \tilde{\boldsymbol{Y}} \right\|_F^2, \tag{33}$$

where $\tilde{\boldsymbol{Y}}$ are the standardized targets (Section 7.5) and $\boldsymbol{\theta}$ collects all weights and biases.

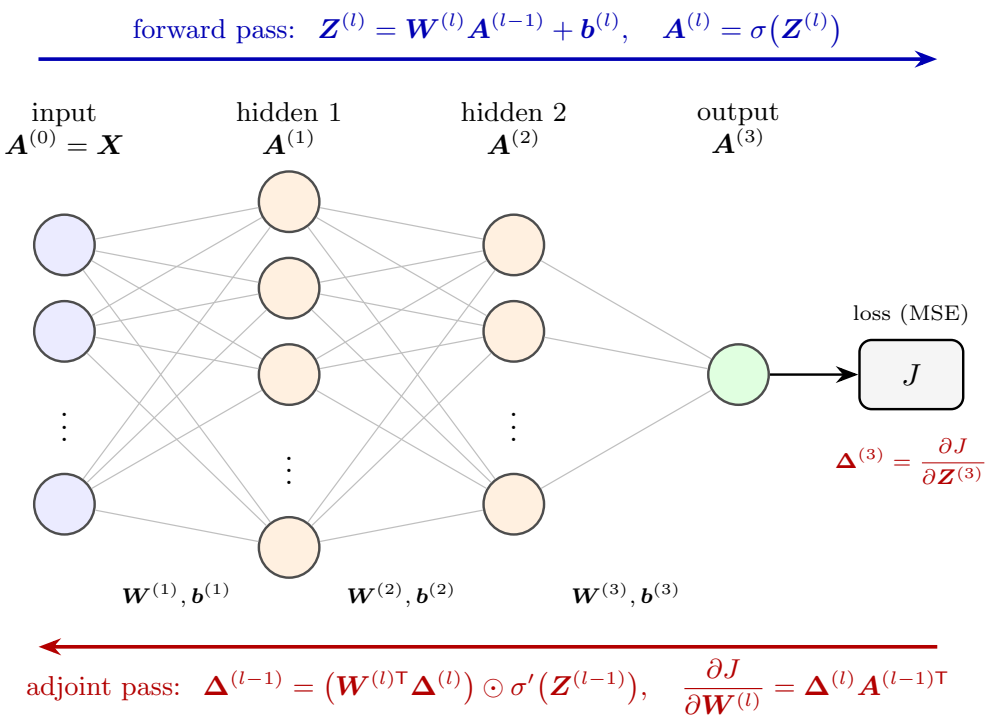


Figure 12: Fully connected network with forward pass (left to right) and adjoint pass (right to left).

7.3 Gradient by the Adjoint Method

The gradient of J is computed by reverse-mode differentiation, known in the neural network literature as backpropagation [5] and in the optimization literature as the discrete adjoint method [1]. For each layer the adjoint variable is the sensitivity of the loss with respect to the pre-activation,

$$\boldsymbol{\Delta}^{(l)} = \frac{\partial J}{\partial \boldsymbol{Z}^{(l)}}. \tag{34}$$

It is initialised at the linear output layer by differentiating Eq. (33),

$$\boldsymbol{\Delta}^{(L)} = \frac{2}{n_L N_b}\big(\boldsymbol{A}^{(L)} - \tilde{\boldsymbol{Y}}\big), \tag{35}$$

and propagated backwards with

$$\boldsymbol{\Delta}^{(l-1)} = \Big(\big(\boldsymbol{W}^{(l)}\big)^{\mathsf{T}}\boldsymbol{\Delta}^{(l)}\Big) \odot \sigma'\big(\boldsymbol{Z}^{(l-1)}\big), \qquad l = L, \dots, 2. \tag{36}$$

The parameter gradients of every layer follow directly from the adjoint and the stored activations of the forward pass:

$$\frac{\partial J}{\partial \boldsymbol{W}^{(l)}} = \boldsymbol{\Delta}^{(l)}\big(\boldsymbol{A}^{(l-1)}\big)^{\mathsf{T}}, \qquad \frac{\partial J}{\partial \boldsymbol{b}^{(l)}} = \boldsymbol{\Delta}^{(l)}\mathbf{1}. \tag{37}$$

One backward pass costs a small multiple of one forward pass, independent of P. The gradients are returned in a structure with the same fields as the parameters $(\boldsymbol{W}^{(1)}, \boldsymbol{b}^{(1)}, \dots)$, so they can be passed directly to the same optimizer step functions used in the previous sections.

7.4 Gradient check against central finite differences

To verify the adjoint gradient, it was compared with central finite differences on a small network ($[1, 5, 4, 1]$, tanh, 20 random samples). Every weight and bias θ_j is perturbed by $\pm h$:

$$g_j^{\mathrm{FD}} = \frac{J(\boldsymbol{\theta} + h\boldsymbol{e}_j) - J(\boldsymbol{\theta} - h\boldsymbol{e}_j)}{2h}, \qquad h = 10^{-6}, \tag{38}$$

and the relative error $|g_j^{\mathrm{FD}} - g_j^{\mathrm{adj}}| / \max(1, |g_j^{\mathrm{FD}}|)$ is computed. The largest error over all parameters was 2.44×10^{-9}, well below the threshold of 10^{-5}, so the adjoint implementation is correct.

Table 8: Gradient check: maximum relative error between adjoint and central finite-difference gradients (layers $[1, 5, 4, 1]$, tanh, $h = 10^{-6}$).

Parameter array	**Size**	**Max. relative error** e	**Passed** ($e < 10^{-5}$)
$\boldsymbol{W}^{(1)}$	5×1	1.27×10^{-9}	yes
$\boldsymbol{b}^{(1)}$	5×1	2.44×10^{-9}	yes
$\boldsymbol{W}^{(2)}$	4×5	1.94×10^{-9}	yes
$\boldsymbol{b}^{(2)}$	4×1	1.25×10^{-9}	yes
$\boldsymbol{W}^{(3)}$	1×4	3.07×10^{-10}	yes
$\boldsymbol{b}^{(3)}$	1×1	3.03×10^{-11}	yes

7.5 Training Setup

Data. The samples are split randomly into 70% training, 15% validation and 15% test data. Inputs are scaled to $[-1, 1]$ and outputs are standardized, using training statistics only. A scaled MSE of 1 therefore means the model is no better than predicting the mean. After each epoch the validation loss is computed, the weights with the lowest validation loss are kept, and the test set is used once at the end.

Batch size. The batch size B sets how many samples are used per gradient and update. With $B = N_{\mathrm{tr}}$ the method is full-batch gradient descent (one update per epoch, exact gradient), with $1 < B < N_{\mathrm{tr}}$ it is mini-batch training, and with $B = 1$ it is stochastic gradient descent (SGD). The samples are shuffled every epoch. The results below use full batch.

Optimizers and fairness. ADAM, AdamW [2], AMSGrad [4] and Twin ADAM (Section 6.4) are compared using the same training code, only the step function is exchanged. For Twin ADAM two networks are trained on the same data and coupled as in Section 6.4, which costs two gradients per update. All runs use the same random seed, so data, split and initial weights are identical, and the same hyperparameters (Table 9).

Table 9: Test problems and common settings.

Test problems	Rosenbrock fit with $D = 1, 2, 100$ inputs; real-world data set Concrete Compressive Strength (UCI)
Network (100D)	$[100, 128, 64, 1]$, $P = 21\,249$, $N = 100\,000$ samples in $[-2, 2]^{100}$
Activation	tanh (hidden), linear (output)
Hyperparameters	$\alpha = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, AdamW $\lambda = 0.1$
Batch size	full batch ($B = N_{\mathrm{tr}}$)
Seed	1, reset before every run

Algorithm 3 ANN training with an exchangeable ADAM-type optimizer (`ADAM_ANN.m`)

Require: Data $(\boldsymbol{X}, \boldsymbol{Y})$, layer widths, activation σ, epochs E, batch size B, step function `opt`, hyperparameters $(\alpha, \beta_1, \beta_2, \varepsilon)$

Split data 70/15/15; scale inputs to $[-1, 1]$, standardize outputs (training statistics)
Initialise $\boldsymbol{\theta}$; optimizer state $\leftarrow \emptyset$; $t \leftarrow 0$; $J_{\text{va}}^{\text{best}} \leftarrow \infty$
for $e = 1, \ldots, E$ **do**
    Shuffle the training samples
    for each batch $\mathcal{B}$ of size B **do**
        $\boldsymbol{A}^{(L)} \leftarrow$ forward pass on $\mathcal{B}$ ▷ store $\boldsymbol{Z}^{(l)}, \boldsymbol{A}^{(l)}$
        $\nabla_{\boldsymbol{\theta}} J \leftarrow$ adjoint pass on $\mathcal{B}$ ▷ Section 7.3
        $t \leftarrow t + 1$
        $[\boldsymbol{\theta}, \text{state}] \leftarrow \texttt{opt}(\boldsymbol{\theta}, \nabla_{\boldsymbol{\theta}} J, \text{state}, t, \text{hp})$ ▷ ADAM, AdamW, AMSGrad
    end for
    Evaluate J_{tr} and J_{va} on the complete sets
    if $J_{\text{va}} < J_{\text{va}}^{\text{best}}$ **then**
        $J_{\text{va}}^{\text{best}} \leftarrow J_{\text{va}}$; $\boldsymbol{\theta}^{\text{best}} \leftarrow \boldsymbol{\theta}$
    end if
end for
Evaluate J_{te} with $\boldsymbol{\theta}^{\text{best}}$
return $\boldsymbol{\theta}^{\text{best}}$, loss histories, J_{te}

Evaluation criteria. Since the minimum of the training loss is unknown, the optimizers are compared by the quality of the trained model and the cost of training:

- **Test MSE (scaled)** with the best-validation parameters. This is the main criterion, since it measures prediction on data not used in training.
- **Training MSE and best validation MSE**, to show how well the data is fitted.
- **Generalisation ratio** $G = J_{\text{te}}/J_{\text{tr}}$. Values close to 1 mean the model generalises; values much larger than 1 indicate overfitting.
- **Cost**: wall-clock time and the number of gradient evaluations. With E epochs this is

$$C \approx 6\, P\, k\, E\, N_{\text{tr}}, \tag{39}$$

  with $k = 1$ for ADAM, AdamW and AMSGrad and $k = 2$ for Twin ADAM. With the same P, E and N_{tr} for all optimizers, Twin ADAM therefore has twice the cost of the other methods, which is consistent with the measured times.

7.6 Twin ADAM for Network Training

The twin-gradient step-length modulation of Section 6.4 [3] was transferred to the network. Two networks with independent random initialisations, $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$, are trained on the same batches. Both compute their own ADAM step $\boldsymbol{s}_1, \boldsymbol{s}_2$, and the multipliers γ_1, γ_2 are obtained from the same 2×2 system as in Section 6.4, with all weights and biases of a network treated as one vector:

$$\min_{\gamma_1, \gamma_2} \ \big\|(\boldsymbol{\theta}_1 + \gamma_1 \boldsymbol{s}_1) - (\boldsymbol{\theta}_2 + \gamma_2 \boldsymbol{s}_2)\big\|_2^2, \qquad \gamma_{1,2} \in [0.5,\, 3.5]. \tag{40}$$

If the two steps are nearly parallel the system is close to singular, and $\gamma_1 = \gamma_2 = 1$ is used, which reduces to two independent ADAM runs. After each epoch both networks are evaluated on the validation set, and the better one is kept as the current best model. Each update needs one forward and one adjoint pass for each network, i.e. two gradient evaluations instead of one.

7.7 Results

Table 10 summarises all ANN benchmarks.

Table 10: ANN benchmark results (scaled MSE, seed 1, settings of Table 9).

Optimizer	Train MSE	Best val. MSE	Test MSE	G	Time [s]
*sin(0.2x) + 0.05*randn(1,200)*					
ADAM	4.67×10^{-3}	1.32×10^{-2}	1.20×10^{-2}	2.56	2.99
AdamW	1.64×10^{-1}	1.15×10^{-1}	1.18×10^{-1}	0.72	3.80
AMSGrad	2.34×10^{-1}	3.24×10^{-1}	3.19×10^{-1}	1.37	3.91
Twin ADAM	9.42×10^{-3}	1.36×10^{-2}	1.30×10^{-2}	1.38	6.99
Rosenbrock 2D					
ADAM	5.16×10^{-5}	1.48×10^{-4}	4.70×10^{-5}	0.91	1.99
AdamW	2.21×10^{-3}	5.04×10^{-3}	2.54×10^{-3}	1.14	1.98
AMSGrad	1.02×10^{-4}	3.07×10^{-4}	9.36×10^{-5}	0.92	1.95
Twin ADAM	8.31×10^{-5}	2.50×10^{-4}	8.39×10^{-5}	1.0	3.2
Rosenbrock 100D					
ADAM	2.01×10^{-1}	5.44×10^{-1}	5.44×10^{-1}	2.70	493.18
AdamW	2.95×10^{-1}	6.27×10^{-1}	6.44×10^{-1}	2.18	531.00
AMSGrad	1.94×10^{-1}	5.26×10^{-1}	5.23×10^{-1}	2.69	573.19
Twin ADAM	5.21×10^{-1}	7.12×10^{-1}	7.25×10^{-1}	1.39	950.16
Real-world data set: Concrete dataset					
ADAM	4.68×10^{-2}	9.23×10^{-2}	1.15×10^{-1}	2.46	3.37
AdamW	4.20×10^{-2}	7.61×10^{-2}	1.13×10^{-1}	2.71	3.34
AMSGrad	8.51×10^{-2}	1.05×10^{-1}	1.42×10^{-1}	1.67	3.56
Twin ADAM	4.83×10^{-2}	7.48×10^{-2}	1.53×10^{-1}	3.17	6.04

7.7.1 Low-dimensional Curve fits ($D = 1$, $D = 2$)

Figure 13 compares the learned function with the exact sin(0.2x) function with 0.5% noise. The prediction follows the exact curve with a scaled test MSE of [**value**]. For the 2D Rosenbrock the prediction reproduces the curved valley. These two cases show that the complete pipeline works before it is applied in high dimension. The differences between the optimizers are [**small / large**], see Table 10.

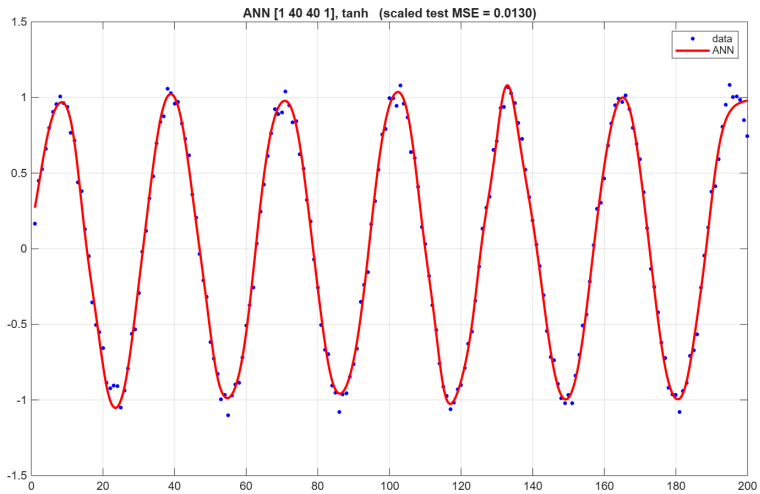
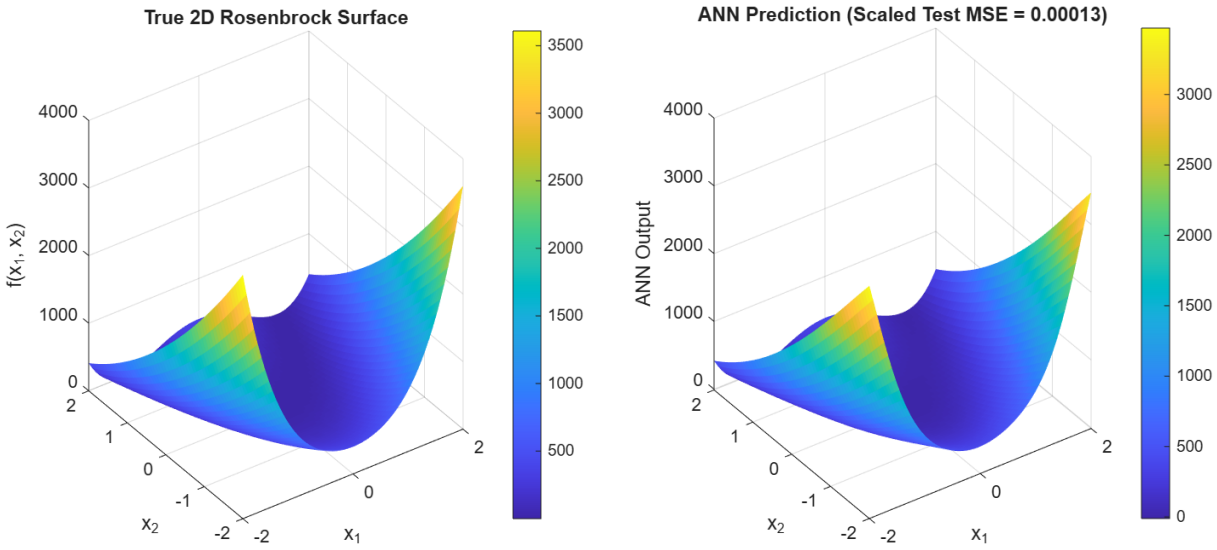


Figure 13: ANN approximation of the for a 1D function sin(0.2x) with 0.5% noise (ADAM, settings of Table 9).

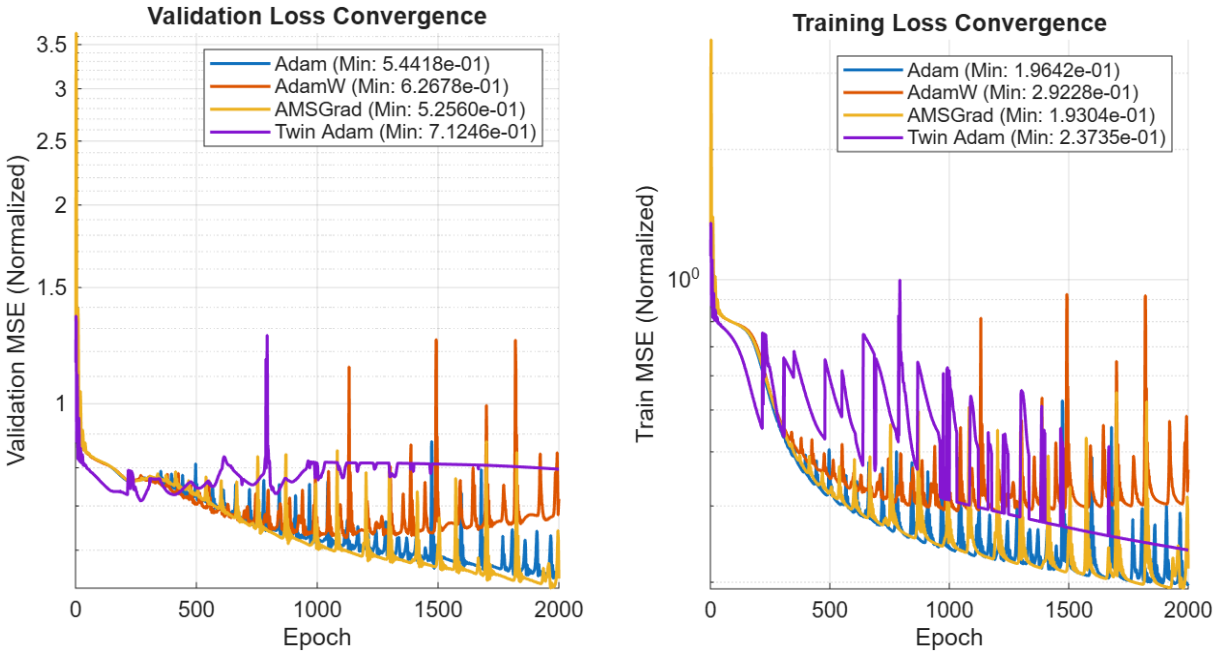


(a) Original Function **(b)** ANN prediction

Figure 14: ANN approximation of the 2D Rosenbrock function (ADAM, settings of Table 9).

7.7.2 100-dimensional Rosenbrock ($D = 100$, $P = 21\,249$)

Figure 15 shows the validation and training loss of the four optimizers over the epochs. [**Describe the curves: which optimizer decreases fastest in the first epochs, which one reaches the lowest validation loss, whether any curve stagnates or oscillates.**] The lowest test MSE was obtained by [**optimizer**] with [**value**]. The generalisation ratio was between [**min**] and [**max**] for all optimizers, which [**does / does not**] indicate overfitting with $N = 100\,000$ samples. Figure **??** compares predicted and true values on the test set.



(a) Validation loss **(b)** Training loss

Figure 15: Convergence of ADAM, AdamW, AMSGrad and Twin ADAM for the 100-dimensional Rosenbrock regression.

7.7.3 Real-world data set.

As a test on measured data, the Concrete Compressive Strength data set from the UCI Machine Learning Repository was used [6]. It contains 1030 laboratory tests without missing values. Each sample has 8 inputs: the amounts of cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate and fine aggregate (all in $\mathrm{kg/m^3}$ of mixture) and the age of the concrete in days. The output is the compressive strength in MPa. The relation between mixture and strength is strongly nonlinear, which is why the data set was originally used to test neural networks [6]. Unlike the Rosenbrock fits, the data contain measurement scatter, and with $N = 1030$ the training set (721 samples) is small compared to the number of network parameters, so overfitting becomes possible. The same split, scaling and settings as for the other problems were used. **[Results: test MSE of the optimizers, Table 10, and Figure 16.]**

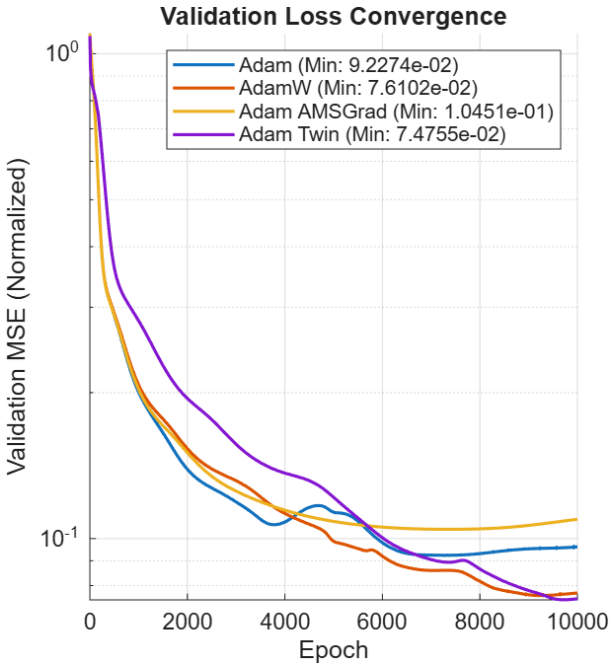


Figure 16: Validation loss of ADAM, AdamW, AMSGrad and Twin ADAM over the epochs for the Concrete Compressive Strength data set.

7.7.4 Twin ADAM

In the ANN benchmarks Twin ADAM did not reach a lower validation loss than ADAM within the same number of epochs **[confirm with final numbers]**. Since it computes two gradients per update, its wall time per epoch was **[x]** times that of ADAM (Figure 17). This differs from the direct 100-dimensional Rosenbrock problem (Section 6.4), where the twin coupling could help when the learning rate was chosen too small.

A likely reason is the structure of the network loss. The hidden neurons of a layer can be permuted, together with their weights, without changing the output of the network. The loss therefore has many equivalent minima at different locations in parameter space. Two independently initialised networks generally move towards different ones of these minima, so the direction that brings $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$ closer together is not a direction that reduces the loss. The step multipliers then mainly act as a change of the learning rate. On problems with a single basin, such as the Rosenbrock function itself, the distance between the two iterates carries information about the position of the minimum, which is the situation the method was designed for [3].

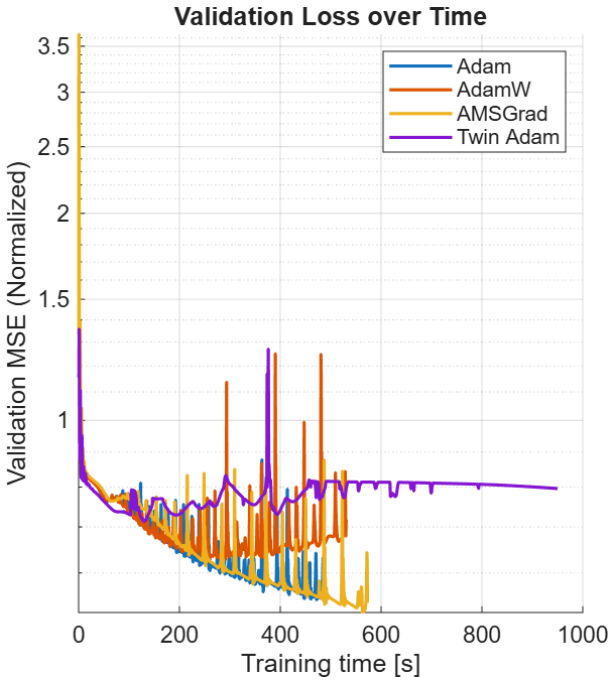


Figure 17: ADAM and Twin ADAM compared at equal computing time (100-dimensional Rosenbrock regression).

7.8 Discussion

The ANN experiments extend the benchmarks of Sections 3 to 6 from at most 100 to 21 249 unknowns. The adjoint method gave gradients that agreed with central finite differences to [**order of** e], at the cost of one backward pass instead of $2P$ loss evaluations. [**Summarise the main ranking of ADAM, AdamW and AMSGrad over the test problems in one or two sentences, and whether the ranking from the direct Rosenbrock problem in Section 5.4 carries over to network training.**] Twin ADAM doubled the cost per update without improving the result on the networks, while it can help on the single-basin problems of Section 6.4.

All runs use one fixed seed and the same hyperparameters, so the comparison isolates the effect of the update rule. Repeated runs with other seeds and a study of the learning rate would show how sensitive the ranking is to these choices.

References

[1] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.

[2] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization (AdamW). *arXiv preprint arXiv:1711.05101*, 2017.

[3] Anna De Magistris, Michiel E. Hochstenbach, and Gerardo Toraldo. A twin gradient method for unconstrained optimization. *arXiv preprint arXiv:2607.11575*, 2026.

[4] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.

[5] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[6] I-Cheng Yeh. Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12):1797–1808, 1998. Data set: UCI Machine Learning Repository, `https://doi.org/10.24432/C5PK67`.